

Universidad de Monterrey
Materia: Bases de Datos Avanzadas
Maestro: Raúl Morales Salcedo



Documento Técnico Final:
MediNFC

Código de honor: “Declaramos que este proyecto es resultado de nuestro trabajo original. No hemos incurrido en plagio ni en el uso indebido de recursos externos sin citar. Cualquier apoyo recibido ha sido debidamente reconocido. Nos comprometemos a actuar con integridad académica conforme a los principios de la UDEM.”

Alumnos:

Katya María Treviño Treviño #627463 ITC 4to semestre

Ricardo Salido Navarro #626473 ITC 4to semestre

Juan Pablo Oranday #625949 ITC 4to semestre

Vicente Salazar Astorga #656072 ITC 6to semestre

Fecha: 11 de mayo del 2026

Índice

1. Introducción	4
1.1 Importancia del sistema	5
2. Planteamiento del problema	5
2.1 Descripción técnica	5
2.2 Alcance	6
3. Arquitectura del sistema	7
3.1 Backend (Flask)	8
3.2 Bases de datos (PostgreSQL y MongoDB)	9
3.3 Frontend	9
3.4 Integración con dispositivos IoT	10
4. Modelo de datos	11
4.1 Modelo relacional final	11
4.2 Integración NoSQL	12
5. Implementación de base de datos	13
5.1 Stored Procedures	13
5.2 Triggers	17
5.3 Views	18

6. Analítica y KPI	19
6.1 Definición de indicadores	19
6.2 Interpretación	20
7. Simulaciones y escenarios	20
7.1 Casos de uso reales	20
7.2 Ejecución del sistema	21
7.3 Resultados obtenidos	22
8. Visualización de datos	23
8.1 Uso de Highcharts	23
8.2 Explicación de dashboards	23
9. Integración PostgreSQL y MongoDB	24
9.1 Arquitectura híbrida	24
9.2 Flujo de datos	25
9.3 Justificación técnica	26
10. Resultados	27
10.1 Impacto del sistema	27
10.2 Beneficios obtenidos	27

11. Conclusiones

27

11.1 Aprendizajes

27

11.2 Limitaciones

28

11.3 Trabajo futuro

28

12. Referencias

29

1. Introducción

El aumento de personas adultas mayores en México y en el mundo representa un reto importante para los sistemas de salud y para las familias que cuidan de ellos. Según la Organización Mundial de la Salud, cerca del 50 % de los pacientes con enfermedades crónicas no toma sus medicamentos como se les indicó. Esto puede causar más hospitalizaciones, complicaciones que se pudieron evitar e incluso muertes antes de tiempo.

En los adultos mayores, este problema puede ser más grave porque muchas veces tienen dificultades para recordar, moverse o distinguir bien los medicamentos. Por ejemplo, puede costarles trabajo acordarse de la hora exacta en la que deben tomar una pastilla, diferenciar entre medicamentos parecidos o avisarle a su cuidador cuando ocurre algún problema.

La falta de seguimiento en la toma de medicamentos no solo afecta la salud del paciente, también complica el trabajo de los cuidadores. Muchas veces, los familiares o cuidadores profesionales no tienen una herramienta digital que les ayude a registrar de forma clara cuándo, dónde y cómo se tomó cada dosis. Además, cuando el médico recibe esta información, normalmente llega incompleta, en hojas, mensajes o comentarios sueltos, lo que dificulta tomar buenas decisiones sobre el tratamiento.

Por eso, es necesario crear un sistema que ayude a identificar correctamente cada medicamento, confirmar que el cuidador esté cerca del paciente al momento de administrar la dosis y facilitar la comunicación entre médico, cuidador y administrador. De esta manera, se tendría un registro completo y confiable que ayudaría a revisar el tratamiento y hacer cambios cuando sea necesario.

1.1 Importancia del sistema

MediNFC fue creado para responder a esta necesidad. Es una plataforma web que usa etiquetas NFC pegadas en cada empaque de medicamento y dispositivos GPS asignados a los cuidadores. Con esto, el sistema puede registrar de forma automática cada toma, indicando la hora, la ubicación y si el cuidador estaba cerca del paciente.

Esta combinación permite saber no solo si el medicamento fue tomado, sino también si se tomó dentro del horario correcto y si realmente ocurrió cerca del paciente.

La importancia de MediNFC se basa en tres puntos principales. **Primero**, ayuda a mejorar la seguridad del paciente, ya que puede detectar dosis repetidas, dosis olvidadas y tomas hechas fuera de tiempo mediante reglas dentro de la base de datos. **Segundo**, le da al médico un historial más claro del comportamiento del paciente, lo que permite ajustar el tratamiento usando información real. **Tercero**, facilita el trabajo del cuidador, porque solo necesita acercar el teléfono a una etiqueta NFC, mientras el sistema se encarga de guardar, revisar y reportar cada registro de manera confiable.

2. Planteamiento del problema

2.1 Descripción técnica

Los sistemas normales para controlar la toma de medicamentos tienen tres problemas importantes que MediNFC busca solucionar.

El **primero** es que no siempre se puede comprobar si el cuidador realmente está con el paciente cuando registra la toma. En muchas aplicaciones, cualquier persona puede marcar que el medicamento ya fue tomado, aunque no esté presente. El **segundo** problema es que no siempre se identifica bien el medicamento. Si el cuidador tiene que escribir el nombre o escogerlo de una lista, puede equivocarse, y ese error puede afectar el tratamiento del paciente. El **tercer** problema es que muchos sistemas no revisan automáticamente los datos para detectar situaciones peligrosas, como cuando se registra la misma medicina dos veces en muy poco tiempo o fuera del horario recomendado. Además, la información médica que se genera casi nunca se usa para analizar patrones o detectar riesgos. Muchas veces se queda en hojas de papel, archivos separados o tablas de Excel, sin un registro claro y completo. Esto dificulta que el médico pueda identificar a tiempo si un

paciente está en riesgo y hace que las decisiones se tomen cuando el problema ya ocurrió, en lugar de prevenirlo.

MediNFC busca resolver estos problemas usando tres elementos principales. El **primero** son las etiquetas NFC, que tienen un identificador único y se colocan en cada medicamento. Cuando el cuidador acerca el celular a la etiqueta, el sistema puede saber exactamente qué medicamento es y si pertenece a la receta activa del paciente. El **segundo** elemento es la ubicación del cuidador. El sistema usa GPS y la ubicación del navegador para comprobar si el cuidador está cerca del domicilio del paciente al momento de registrar la toma. Esto ayuda a confirmar que la dosis se registró en el lugar correcto. El **tercer** elemento son reglas dentro de la base de datos. Estas reglas revisan cada registro en tiempo real para detectar dosis repetidas, tomas tardías o registros fuera del horario permitido. Cuando algo no cumple con las reglas, el sistema genera una alerta y guarda el evento en una bitácora.

2.2 Alcance

El sistema cubre todo el proceso relacionado con la toma correcta de medicamentos, desde que el médico hace la receta hasta que se generan los reportes clínicos. Su alcance incluye los siguientes módulos:

- **Gestión de información por parte del administrador:** médicos, pacientes, cuidadores, medicamentos, diagnósticos, especialidades, beacons y dispositivos GPS.
- **Recetas creadas por el médico:** el médico puede crear recetas con uno o varios medicamentos, indicar la dosis, la frecuencia, la hora de la primera toma y el tiempo de tolerancia. También puede cancelar o volver a emitir recetas cuando sea necesario ajustar el tratamiento.
- **Registro de tomas por parte del cuidador:** el cuidador puede escanear la etiqueta NFC desde el navegador, mientras el sistema registra automáticamente la ubicación, revisa si está cerca del paciente y permite atender alertas con observaciones clínicas.
- **Agenda diaria del paciente:** el sistema genera las tomas que el paciente debe realizar cada día con base en la receta activa.

- **Detección automática de problemas:** el sistema clasifica las tomas como correctas, tardías, duplicadas u omitidas, y genera alertas según el caso.
- **Revisión de tomas omitidas:** cada cinco minutos se ejecuta un proceso automático que revisa si hay medicamentos que no fueron registrados a tiempo.
- **Reportes clínicos y operativos:** se pueden consultar reportes de adherencia por paciente, médico y cuidador, además de tendencias, riesgos de omisiones seguidas, mejoras, supervisión general, historial de tomas y mapa de proximidad.
- **Dashboards y gráficas:** el sistema muestra información visual mediante gráficas y paneles, usando MongoDB para consultar los datos más rápido sin cargar demasiado la base de datos principal.
- **Registro completo de actividades:** se guarda una bitácora de reglas del sistema, cambios realizados en tablas importantes, accesos en PostgreSQL y registros en MongoDB con eliminación automática después de cierto tiempo.

Quedan fuera del alcance del sistema la facturación, la conexión con expedientes clínicos externos, el uso de estándares como HL7/FHIR y una aplicación móvil nativa. MediNFC funciona como una aplicación web adaptable a computadora y celular, probada en navegadores Chrome y Edge. Para escanear las etiquetas NFC se necesita usar **Chrome en Android**, ya que actualmente es el navegador compatible con la Web NFC API.

3. Arquitectura del sistema

MediNFC se creó como una aplicación web integrada, basada en el modelo cliente-servidor. Está organizada en tres partes principales: presentación, aplicación y datos.

La parte de **presentación** es lo que ve el usuario en el navegador. Se muestra con páginas HTML generadas desde el servidor y se complementa con JavaScript para que algunas funciones sean más dinámicas. La parte de **aplicación** funciona con Flask. Aquí se controla el inicio de sesión, los permisos según el tipo de usuario y la

conexión con la base de datos. La parte de **datos** usa dos bases de datos. PostgreSQL guarda la información principal y más importante del sistema, como recetas, pacientes, medicamentos y registros de tomas. MongoDB se usa para guardar información de apoyo, como datos para reportes, historiales grandes y bitácoras menos importantes. Además, el sistema se conecta con Traccar Server, un servicio externo que permite dar seguimiento GPS continuo a los cuidadores.

3.1 Backend (Flask)

El backend de MediNFC está hecho en **Python** usando **Flask 3.1.3**. No se usaron Blueprints, por lo que la aplicación se mantiene en un solo proyecto con todas las rutas registradas en el archivo app.py. Los controladores están organizados por tipo de usuario dentro de la carpeta controllers/. Esta decisión ayuda a que el proyecto sea más fácil de instalar, entender y presentar, sin perder el orden entre los diferentes roles del sistema.

Para conectarse con la base de datos se usa la librería **psycopg 3.3.3**, sin usar un ORM. Esto se hizo a propósito, ya que permite aprovechar mejor las funciones propias de **PostgreSQL**, como procedimientos almacenados, triggers, vistas y cursores. Toda la lectura, escritura y generación de reportes se hace por medio de procedimientos almacenados. Solo hay dos excepciones: la verificación de contraseña con bcrypt durante el inicio de sesión y el endpoint/api/gps, que recibe datos GPS mediante el protocolo OsmAnd.

El inicio de sesión se maneja con **Flask-Bcrypt 1.0.1**, que sirve para proteger las contraseñas, y con las sesiones normales de Flask para mantener al usuario activo dentro del sistema. Cada sesión guarda cinco datos: user_id, rol, id_rol, nombre y foto_perfil. Para controlar el acceso se usan dos decoradores: login_requerido, que revisa si el usuario ya inició sesión, y rol_requerido, que permite entrar solo a las rutas autorizadas según el rol del usuario.

El usuario administrador funciona de forma distinta, ya que sus datos no están guardados en la tabla usuario, sino directamente en el archivo config.py. Esto ayuda a iniciar el sistema más fácilmente desde el principio.

Para las tareas automáticas se usa **APScheduler 3.11.2**, que se ejecuta dentro de la aplicación. Este proceso corre cada cinco minutos y ejecuta **sp_detectar_omisiones**. Su función es revisar la agenda de medicamentos y marcar como omitidas las tomas que ya pasaron del tiempo permitido. Cuando detecta una omisión, genera una alerta de prioridad alta sin que alguien tenga que hacerlo manualmente.

La conexión con **MongoDB** está en el archivo `mongo_client.py` usando **pymongo 4.10.1**. Las funciones que escriben datos en MongoDB están preparadas para no detener el sistema si ocurre un error. Es decir, si MongoDB falla o no responde, el error se guarda en el log, pero la aplicación principal sigue funcionando. Esto evita que un problema en los registros o reportes afecte el funcionamiento clínico del sistema.

3.2 Bases de datos (PostgreSQL y MongoDB)

La base de datos principal de MediNFC se llama **medi_nfc2** y está hecha en **PostgreSQL**. Contiene **33 tablas** organizadas en cinco grupos principales: catálogos, información clínica, recetas, eventos del sistema y registros de seguimiento. Se eligió PostgreSQL porque permite trabajar bien con procedimientos almacenados, triggers, tipos de datos especiales, restricciones y reglas más completas dentro de la base de datos. En esta base se crearon más de **60 procedimientos almacenados**, los cuales sirven para manejar catálogos, operaciones clínicas, etiquetas NFC y reportes. Todos siguen una misma forma de trabajo: usan parámetros para indicar si la acción fue correcta, mostrar un mensaje de resultado y devolver información cuando sea necesario. También usan códigos de respuesta. Por ejemplo, el código **1** significa que todo salió bien, mientras que los valores negativos indican errores, como datos faltantes, errores en las reglas del sistema, registros no encontrados o datos repetidos.

La base de datos secundaria se llama **medinfc_mongo** y está hecha en **MongoDB**. Esta no sustituye a PostgreSQL, sino que lo complementa. Se usa para guardar información que puede ser más flexible o pesada, como documentos clínicos, logs que se eliminan automáticamente después de cierto tiempo, historiales grandes para gráficas y recorridos GPS largos.

3.3 Frontend

La interfaz del sistema se hizo con plantillas **Jinja2** que Flask muestra en el navegador. Todas las páginas parten de un archivo principal llamado **base.html**, donde se define la barra de navegación, los mensajes del sistema, los estilos y los archivos de JavaScript.

La barra de navegación cambia según el tipo de usuario. El **administrador** ve accesos a catálogos y reportes generales. El **médico** ve sus pacientes, alertas, reportes de adherencia y mapa. El **cuidador** ve su agenda del día, el escaneo NFC, sus alertas y su historial.

Para la parte visual se usan tres librerías externas. **Highcharts** se usa para mostrar gráficas y dashboards, como tendencias, ranking de mejora y riesgo de omisiones. **Leaflet**, junto con mapas de OpenStreetMap, se usa para mostrar la ubicación de cuidadores y pacientes en el mapa. **Lucide Icons** se usa para mostrar íconos más claros y ordenados en lugar de emojis.

La interacción del usuario usa dos funciones importantes del navegador. La primera es la **geolocalización**, que permite obtener la ubicación del cuidador cuando registra una toma o consulta su posición actual. La segunda es la **Web NFC API**, que permite leer la etiqueta NFC del medicamento al acercarlo al teléfono al empaque. Esta función se usa en las pantallas de escaneo y registro de nuevas etiquetas NFC.

3.4 Integración con dispositivos IoT

La conexión con dispositivos IoT es una de las **partes más importantes de MediNFC**. El sistema junta 2 elementos diferentes en una misma plataforma: etiquetas NFC y dispositivos GPS registrados en la base de datos.

- **Lectura NFC con Web NFC API:** Las etiquetas NFC no necesitan batería y cada una tiene un identificador único llamado **UID**, que viene desde fábrica y no cambia. El cuidador usa **Chrome en Android**, ya que es el navegador compatible con esta función, y entra a la pantalla de escaneo del paciente. Cuando el cuidador acerca el celular a la etiqueta NFC del medicamento, el sistema lee automáticamente el número de la etiqueta. Después, ese dato se coloca en el formulario y se manda al backend para registrar la toma del medicamento. La pantalla para registrar nuevas etiquetas NFC usa el mismo proceso. Gracias a esto, el cuidador no tiene que escribir manualmente el UID, lo que reduce errores.
- **Seguimiento GPS con Traccar Client:** El seguimiento GPS de los cuidadores se realiza con Traccar Client, una aplicación móvil de código abierto instalada en el celular del cuidador. Esta app manda la ubicación al servidor de MediNFC usando el protocolo **OsmAnd HTTP**. La información que se envía incluye datos como el **IMEI del dispositivo**, latitud, longitud, precisión y hora del registro.

El backend de Flask tiene el endpoint **/api/gps**, que recibe esta información. Después, busca el IMEI en la tabla **gps_imei** para saber a qué cuidador pertenece. Si el dispositivo está registrado, guarda la ubicación en PostgreSQL y también deja un registro en MongoDB. Si el IMEI no está registrado, el sistema no guarda la ubicación y solo deja una advertencia en

el log. Además, algunas pantallas del cuidador también usan la ubicación del navegador mediante **navigator.geolocation**. Esto sirve como una segunda forma de obtener la ubicación. Así, aunque el cuidador no tenga instalada la app Traccar Client, el sistema puede registrar su posición cuando usa la plataforma.

4. Modelo de datos

4.1 Modelo relacional final

El diseño de la base de datos se hizo siguiendo reglas de organización para evitar información repetida y mantener los datos ordenados. En total, el sistema tiene **31 tablas**, divididas en los siguientes grupos:

- **Catálogos:** Las tablas como **diagnostico**, **especialidad**, **via_administracion**, **unidad_dosis**, **tipo_alerta**, **estado_alerta**, **canal_notificacion** y **resultado_validacion** guardan datos que se usan en varias partes del sistema. Por ejemplo, sirven para tener listas fijas de diagnósticos, tipos de alerta, estados o formas de administrar un medicamento. Sus identificadores se mantienen estables para que las reglas del sistema puedan usarlos sin problemas.

- **Entidades clínicas:** Las tablas **medico**, **paciente**, **cuidador** y **usuario** representan a las personas que usan o forman parte del sistema.

La tabla **usuario** está separada de **medico** y **cuidador** porque así se pueden manejar mejor las cuentas, contraseñas, altas y bajas de usuarios.

También existen tablas intermedias como **medico_especialidad**, **paciente_diagnostico** y **paciente_cuidador**, que sirven para relacionar información. Por ejemplo, un paciente puede tener varios diagnósticos o varios cuidadores.

En el caso de **paciente_cuidador**, se puede indicar si un cuidador es el principal y si sigue activo. Esto permite cambiar cuidadores sin perder el historial anterior.

- **Prescripciones:** La tabla **receta** guarda la información general de la receta médica.

La tabla **receta_medicamento** guarda el detalle de cada medicamento dentro de esa receta. Ahí se define qué medicamento debe tomar el paciente, la dosis, la frecuencia, el tiempo de tolerancia y la hora de la primera toma.

Con esta información, el sistema genera la agenda_toma, que indica las tomas que debe seguir el paciente durante los días de la receta.

- **Eventos operativos:** Las tablas evento_nfc, evento_proximidad, agenda_toma y alerta guardan lo que pasa durante el uso real del sistema. **evento_nfc** registra cada vez que se lee una etiqueta NFC, junto con la fecha y hora.

evento_proximidad guarda la distancia calculada entre el cuidador y la ubicación del paciente, además de indicar si la cercanía fue válida o no. Cuando se registra un evento NFC, se activan reglas en la base de datos que revisan si la toma fue correcta, tardía, duplicada u omitida. Si algo no cumple, el sistema genera una alerta y guarda el registro correspondiente.

- **Trazabilidad:** Las tablas log_acceso, audit_cambios y bitacora_regla_negocio sirven para llevar un control de lo que ocurre dentro del sistema.

log_acceso guarda los intentos de inicio de sesión, tanto correctos como fallidos.

audit_cambios registra los cambios hechos en tablas importantes, guardando el valor anterior y el nuevo.

bitacora_regla_negocio guarda el resultado de las reglas que se aplican automáticamente en la base de datos.

También existe la tabla **nfc_pendiente**, que guarda los UUIDs de etiquetas NFC que ya fueron leídos, pero que todavía no han sido vinculados a un medicamento.

- **Dispositivos IoT:** Las tablas gps_imei, ubicacion_gps y etiqueta_nfc manejan la parte de dispositivos conectados.

gps_imei guarda el dispositivo GPS asignado a cada cuidador, usando su IMEI único.

ubicacion_gps almacena las coordenadas enviadas por Traccar o por el navegador.

etiqueta_nfc relaciona cada UUID de una etiqueta NFC con un medicamento específico dentro de una receta.

4.2 Integración NoSQL

La base de datos MongoDB medinfcmongo se conecta con PostgreSQL usando identificadores compartidos.

Cada documento de MongoDB que necesita relacionarse con información de PostgreSQL incluye un campo con el prefijo pg. Algunos ejemplos son: pg_id_paciente, pg_id_medico, pg_id_cuidador, pg_id_evento, pg_id_alerta y pg_id_usuario.

Esta forma de nombrarlos ayuda a saber fácilmente a qué registro de PostgreSQL pertenece cada documento. Así, cuando se necesita cruzar información entre ambas bases de datos, se puede hacer usando un solo número de identificación, sin tener que copiar datos descriptivos dentro de MongoDB.

Las colecciones se diseñaron en función del caso de uso:

logs_acceso	Login y logout con resultado, IP y user agent	90 días
logs_sistema	Errores y warnings de Flask	30 días
logs_nfc_fallidos	Escaneos NFC rechazados	Sin TTL
historico_adherencia	Métricas diarias por paciente	Sin TTL
eventos_nfc_rt	Eventos NFC desnormalizados para lectura rápida	Sin TTL
alertas_rt	Alertas activas para badge del menú	Sin TTL
ubicaciones_gps_hist	Trayecto GPS capturado por el navegador	Sin TTL
historial_gps	Pings GPS recibidos vía Traccar y OsmAnd	Sin TTL

5. Implementación de base de datos

5.1 Stored Procedures

Los **procedimientos almacenados** son la forma principal en la que Flask se comunica con la base de datos. Esto se hizo para que las reglas del sistema estén en un solo lugar, sean más fáciles de reutilizar y ayuden a evitar errores o consultas inseguras desde la aplicación.

También permite que, si en el futuro se crea una app móvil, una API o un reporte externo, todos puedan usar las mismas reglas de la base de datos sin tener que volver a programarlas desde cero.

Todos los procedimientos almacenados siguen una misma estructura. Cada uno devuelve un resultado que indica si la acción salió bien o si ocurrió algún error. Para eso se usan dos valores principales: **p_ok**, que indica el código del resultado, y **p_msg**, que muestra un mensaje explicando lo que pasó.

También se usa **io_cursor**, que sirve para devolver listas de información cuando el procedimiento necesita mostrar varios registros.

Los códigos de respuesta tienen un significado específico. Por ejemplo, **1** significa que la acción fue exitosa. Los valores negativos indican errores, como datos obligatorios faltantes, reglas del sistema que no se cumplen, registros no encontrados, datos repetidos, acciones no válidas o errores inesperados. Esta forma de manejar los resultados ayuda a que el sistema muestre mensajes claros al usuario.

Los procedimientos almacenados se organizan según su función. A continuación, se muestra una parte de cada categoría:

SP de gestión (CRUD)

sp_gestion_medico	CRUD de médicos
sp_gestion_cuidador	CRUD de cuidadores
sp_gestion_paciente	CRUD de pacientes
sp_gestion_medicamento	CRUD de medicamentos
sp_gestion_diagnostico	CRUD de diagnósticos
sp_gestion_especialidad	CRUD de especialidades
sp_gestion_gps	CRUD de dispositivos GPS / IMEI
sp_gestion_etiqueta_nfc	Alta/baja de etiquetas NFC
sp_gestion_horario	Alta/eliminación de horarios de tomas
sp_gestion_usuario	Gestión de cuentas de usuario

SP operativos

sp_crear_receta	Crea una receta nueva para un paciente.
sp_agregar_receta_med	Añade un medicamento a una receta existente.
sp_cancelar_receta	Cancela una receta vigente.
sp_asignar_cuidador	Asigna cuidador a paciente.
sp_desasignar_cuidador	Desvincula cuidador de paciente.
sp_asignar_diagnostico	Asocia diagnóstico a paciente.
sp_asignar_especialidad	Asocia especialidad a médico
sp_marcar_alerta_atendida	Marca una alerta como atendida.

sp_detectar_omisiones	Detecta y marca tomas omitidas.
-----------------------	---------------------------------

SP específicos de NFC

sp_registrar_toma_nfc	Registra un escaneo NFC como toma de medicamento
sp_nfc_escaneo_desconocido	Registra escaneo de UID no vinculado
sp_nfc_medicamentos_sin_vincular	Lista medicamentos pendientes de vincular
sp_nfc_vincular	Vincula un UID a receta y medicamento

SP de reportes

sp_rep_dashboard_cuidador	KPIs del dashboard del cuidador.
sp_rep_agenda_dia_cuidador	Agenda de tomas del día.
sp_rep_alertas_medico	Alertas pendientes del médico.
sp_rep_alertas_cuidador	Alertas pendientes del cuidador.
sp_rep_badge_alertas	Conteo de alertas para badge en navbar.
sp_rep_perfil_paciente	Datos completos de un paciente.
sp_rep_historial_tomas	Historial de tomas NFC.
sp_rep_grafica_tomas	Serie temporal de tomas para Highcharts.
sp_rep_adherencia_pacientes_medi	% adherencia por paciente.
sp_rep_adherencia_medicos	% adherencia agrupado por médico.
sp_rep_adherencia_cuidadores	% adherencia agrupado por cuidador.

sp_rep_tendencia_adherencia	Tendencia con promedio móvil 7d.
sp_rep_riesgo_omision	Rachas de omisión por paciente y medicamento.
sp_rep_ranking_mejora	Ranking de mejora de adherencia.
sp_rep_mapa_medico	Datos de mapa para vista del médico.
sp_rep_supervision	Supervisión global médico-paciente.
sp_rep_dispositivos_iot	Lista de GPS registrados.
sp_rep_bitacora	Bitácora de eventos del sistema.
sp_rep_auditoria	Log de auditoría de cambios.
sp_rep_log_acceso	Historial de accesos.

Entre todos los procedimientos, **sp_registrar_toma_nfc** es uno de los más importantes, porque concentra gran parte de la lógica clínica del sistema. Este procedimiento revisa que el UID de la etiqueta NFC exista y esté activo. Después registra la ubicación del cuidador, calcula la distancia entre el cuidador y el domicilio del paciente, guarda el evento NFC y también registra si hubo cercanía válida o no.

También es importante **sp_detectar_omisiones**, ya que se ejecuta automáticamente cada cierto tiempo con APScheduler. Su función es revisar la agenda de medicamentos y detectar las tomas que no fueron registradas dentro del tiempo permitido. Cuando encuentra una omisión, la marca en el sistema y genera una alerta de prioridad alta.

5.2 Triggers

Los **triggers** sirven para aplicar reglas de forma automática dentro de la base de datos, sin importar desde qué parte del sistema se haga el cambio. En MediNFC se usan principalmente para revisar eventos NFC y para guardar auditorías de cambios.

- **Triggers relacionados con evento_nfc:** Cada vez que se guarda un registro en **evento_nfc**, se activan dos triggers. El primero se ejecuta **antes de guardar el evento**. Su trabajo es calcular la diferencia de minutos entre la hora real de la lectura NFC y la hora en la que estaba programada la toma.

También revisa si la toma está duplicada, es decir, si se registró otra toma del mismo medicamento en un tiempo menor al permitido. Con esa revisión, el sistema clasifica la toma como **exitosa, tardía, duplicada u omitida**.

El segundo trigger se ejecuta **después de guardar el evento**. Si detecta un problema, genera una alerta, por ejemplo: **toma tardía, dosis duplicada o proximidad inválida**. También guarda el detalle en **bitacora_regla_negocio**, indicando qué regla se aplicó y por qué. Esta bitácora funciona como un historial de explicación del sistema. Gracias a ella, se puede revisar después por qué se generó una alerta y bajo qué condiciones ocurrió.

- **Triggers de integridad de receta:** También existe un trigger que evita que un paciente tenga dos recetas activas al mismo tiempo. Antes de crear una nueva receta, la anterior debe estar cancelada. Esto ayuda a evitar errores médicos, como tener dos tratamientos vigentes que puedan causar duplicidad de medicamentos o confusión en las tomas.
- **Triggers de auditoría:** Las tablas principales del sistema tienen triggers que guardan los cambios en **audit_cambios**. Antes de hacer una modificación, la aplicación indica a qué usuario está realizando el cambio. Luego, el trigger toma ese dato y lo guarda junto con la información modificada. Así, el sistema puede saber quién hizo el cambio, qué dato se cambió, cuál era el valor anterior y cuál es el nuevo. Esta información se puede consultar desde el panel del administrador en **/admin/auditoría**.

5.3 Views

El esquema también incluye **vistas analíticas**, que ayudan a preparar la información para los reportes del sistema. Estas vistas sirven para juntar datos que se usan con frecuencia y evitar repetir muchas consultas SQL.

Algunas vistas importantes son las que muestran la información del **dashboard del cuidador** y la **agenda del día**. También hay vistas que juntan datos de alertas con información del paciente, medicamento, tipo de alerta y estado, para que puedan mostrarse fácilmente en las tablas de la interfaz.

Además, existen vistas que calculan promedios de los últimos **7 días** para analizar la tendencia del paciente. Con base en esos datos, el sistema puede clasificar la situación como **inicio, mejora, declive o estable**, comparando periodos consecutivos.

También hay vistas que ayudan a detectar omisiones seguidas de medicamentos por paciente y medicamento. Dependiendo de cuántas omisiones ocurran seguidas, el sistema asigna un nivel de riesgo como **crítico, alto o moderado**.

6. Analítica y KPI

6.1 Definición de indicadores

MediNFC usa varios indicadores clínicos y operativos para medir qué tan bien se está siguiendo el tratamiento y para detectar posibles riesgos a tiempo. Los principales son los siguientes:

- **Porcentaje de adherencia diaria:** Este indicador muestra qué porcentaje de tomas se cumplieron en un día. Se calcula comparando las tomas realizadas contra las tomas cumplidas, tardías y omitidas. Es una de las métricas más importantes del sistema. Si el porcentaje es mayor al 80 %, se considera una buena adherencia. Si está entre 50 % y 80 %, se considera parcial. Si es menor al 50 %, se considera baja. Este dato se muestra en reportes por paciente, médico y cuidador.
- **Promedio móvil de adherencia de 7 días:** Este indicador muestra el promedio de adherencia de los últimos 7 días. Sirve para ver mejor la tendencia del paciente sin que un solo día afecte demasiado el resultado. Con esto, el médico puede saber si el paciente está mejorando, empeorando o manteniéndose igual.
- **Tendencia clínica:** La tendencia clínica compara el promedio reciente con el promedio anterior. Si el resultado sube más de 3 puntos porcentuales, se considera una mejora. Si baja más de 3 puntos porcentuales, se considera declive. Si el cambio es pequeño, se considera estable.
- **Días consecutivos de omisión:** Este indicador cuenta cuántos días seguidos el paciente dejó de tomar al menos una dosis de un medicamento. Si hay 3 o más días seguidos de omisión, el riesgo se considera alto. Si hay 5 o más días seguidos, el riesgo se considera crítico. Este indicador es muy útil porque permite que el médico actúe antes de que el paciente tenga una complicación.

- **Mejora por persona:** Este indicador compara la adherencia reciente con la anterior, agrupando la información por médico y por cuidador. Sirve para saber quiénes han logrado una mejor evolución en sus pacientes y también para detectar dónde se necesita más apoyo o capacitación.
- **Carga de trabajo:** Este indicador muestra cuántos pacientes activos tiene cada médico y cada cuidador. Sirve para repartir mejor el trabajo y evitar que una persona tenga demasiados pacientes a su cargo.

6.2 Interpretación

Los KPI no se revisan por separado, sino que se entienden mejor cuando se comparan entre sí. Por ejemplo, un paciente puede tener 85 % de adherencia diaria, lo cual parece bueno. Pero si su tendencia va en declive y además lleva tres días seguidos sin tomar un medicamento importante, el sistema puede marcarlo como un caso de alto riesgo. Por otro lado, un paciente con 60 % de adherencia podría parecer preocupante, pero si su tendencia va en mejora y no tiene omisiones seguidas, puede indicar que el tratamiento está funcionando mejor después de un ajuste reciente.

La interpretación de estos indicadores se apoya en colores dentro de la interfaz. El color **verde** se usa para resultados favorables, el **amarillo** para situaciones que necesitan atención y el **rojo** para alertas importantes. Esto ayuda al médico a revisar la información más rápido, especialmente cuando tiene muchos pacientes. Así puede identificar fácilmente qué casos necesitan atención primero.

Además, el ranking de mejora y los reportes de riesgo ordenan la información de mayor a menor urgencia, lo que facilita tomar decisiones y actuar a tiempo.

7. Simulaciones y escenarios

7.1 Casos de uso reales

Para probar que el sistema funciona correctamente, se creó un conjunto de datos de ejemplo que representa situaciones clínicas parecidas a la realidad. Se usaron tres pacientes con casos diferentes para revisar distintos comportamientos dentro de MediNFC.

- **Caso 1: Elena Martínez — buena adherencia con mejora:** Elena Martínez es una paciente con Diabetes Tipo 2 e Hipertensión. Es atendida por una cuidadora formal. Su tratamiento incluye metformina cada 12 horas y losartán una vez al día. En los datos de prueba se simulan 30 días de registros NFC, donde la paciente mantiene una adherencia cercana al 80 % y muestra una mejora en los últimos siete días.

Este caso sirve como ejemplo de funcionamiento normal del sistema.

- **Caso 2: Héctor Treviño — adherencia irregular con caída reciente:** Héctor Treviño es un paciente con Insuficiencia Cardíaca y Fibrilación Auricular. Su tratamiento incluye digoxina y es atendido por un familiar. Su comportamiento muestra una adherencia general del 55 %, pero con una caída importante durante la última semana. Por eso, el sistema genera alertas de declive y detecta omisiones seguidas con nivel de riesgo alto.

Este caso ayuda a mostrar cómo los indicadores pueden detectar riesgos antes de que el problema sea más grave.

- **Caso 3: Consuelo Hernández — baja en los últimos diez días:** Consuelo Hernández es una paciente con Enfermedad de Alzheimer y Osteoporosis. Su tratamiento incluye rivastigmina dos veces al día y calcio. Al inicio, la paciente tiene buena adherencia, pero después del día veinte empieza a fallar en sus tomas. Esto genera varias alertas por medicamentos omitidos.

Este caso sirve para probar cómo el sistema detecta cambios negativos en el tratamiento y cómo puede apoyar al médico para hacer ajustes de dosis cuando sea necesario.

7.2 Ejecución del sistema

Las simulaciones se realizan con el script **seed_test_data.sql**. Este script limpia las tablas, vuelve a crear los catálogos y las entidades principales, inserta recetas con fecha de inicio de hace **30 días** y permite que los triggers generen el calendario completo de tomas.

Después, marca las tomas pasadas como pendientes e inserta alrededor de **60 eventos NFC**, organizados según el comportamiento esperado de cada paciente.

Luego, **APScheduler** ejecuta **sp_detectar_omisiones** cada cinco minutos. Este proceso revisa las tomas anteriores que ya pasaron del tiempo permitido y las marca como omitidas, generando las alertas necesarias.

Al final, los datos quedan listos para usarse en la aplicación. El médico puede iniciar sesión y ver pacientes con diferentes niveles de adherencia, alertas pendientes, gráficas de tendencia con datos reales y un ranking de mejora con información útil.

Además, se crearon escenarios específicos para probar cada regla del sistema. El primero revisa si el sistema detecta una **dosis duplicada**, simulando dos lecturas del mismo medicamento en un tiempo muy corto. El segundo prueba el ajuste **de dosis por falta de control**, donde el médico cancela la receta activa y crea una nueva con una frecuencia diferente, para comparar la adherencia antes y después del cambio.

7.3 Resultados obtenidos

Los tres escenarios funcionaron como se esperaba. El sistema identificó correctamente las tomas tardías cuando pasaron del tiempo permitido, marcó como duplicado el segundo registro cuando ocurrió muy cerca del primero y generó alertas de proximidad inválida cuando el GPS mostró que el cuidador estaba fuera del radio permitido.

La detección de omisiones también se ejecutó sin problemas. Las tomas no registradas a tiempo fueron marcadas como omitidas y las alertas aparecieron en los dashboards correspondientes en tiempo real.

Las gráficas de tendencia mostraron claramente las diferencias entre los tres pacientes. En el caso de Elena, se observó una mejora. En Héctor, se vio una caída importante durante la última semana. En Consuelo, se presentó un descenso más prolongado en su adherencia.

El ranking de mejora ordenó correctamente a los cuidadores y médicos según el cambio en la adherencia. Además, los reportes de riesgo colocaron a Héctor y Consuelo en los primeros lugares, con nivel crítico, debido a sus omisiones seguidas.

8. Visualización de datos

8.1 Uso de Highcharts

Highcharts es la librería que se eligió para mostrar las gráficas y dashboards del sistema. Se usa junto con los módulos Highcharts More y Solid Gauge.

Se eligió porque es fácil de configurar, se ve bien visualmente y ofrece varios tipos de gráficas útiles para el proyecto. Por ejemplo, se usan barras horizontales para rankings, líneas para mostrar tendencias, gráficas de dispersión para comparar datos y medidores tipo velocímetro para indicadores importantes.

Cada gráfica se configura con un objeto de JavaScript, donde se define el tipo de gráfica, el fondo, los ejes, las etiquetas, los mensajes emergentes y los colores. También se ajustan detalles visuales como bordes redondeados, etiquetas claras y colores diferentes según el valor mostrado.

Los datos de las gráficas se preparan desde el servidor usando Jinja2. Flask toma listas de Python y las convierte a formato JSON, para que Highcharts pueda leerlas en el navegador.

Por ejemplo, en la gráfica de tendencia se mandan tres listas: las fechas para el eje X, el porcentaje diario de adherencia y el promedio móvil de 7 días. Así, la gráfica puede mostrar tanto los cambios diarios como la tendencia principal del paciente.

8.2 Explicación de dashboards

El sistema tiene dashboards diferentes según el tipo de usuario. Cada uno muestra la información más importante para las decisiones que debe tomar cada rol.

- **Dashboard del médico:** El dashboard del médico se encuentra en /doctor y muestra cuatro secciones principales. La **primera** muestra cuántos pacientes activos tiene a su cargo. La **segunda** muestra el total de alertas pendientes, separadas por tipo. La **tercera** muestra los pacientes con peor adherencia reciente, ordenados de menor a mayor, para que el médico pueda atender primero los casos más importantes. La **cuarta** muestra los pacientes con omisiones seguidas en nivel alto o crítico. También incluye una gráfica de barras horizontales, donde cada paciente aparece con un color según su nivel de adherencia: verde para buen resultado, amarillo para atención y rojo para alerta.

- **Dashboard del cuidador:** El dashboard del cuidador se encuentra en / cuidador y está enfocado en las actividades del día. La pantalla muestra las tomas programadas, incluyendo la hora, el paciente, el medicamento, la dosis y el UID NFC. También tiene un contador de alertas pendientes para que el cuidador vea rápidamente qué casos necesitan atención. Cuando una toma se completa, su tarjeta cambia de color y se mueve al final de la lista. Así, las tomas pendientes siempre quedan arriba y son más fáciles de revisar.
- **Dashboard del administrador:** El dashboard del administrador se encuentra en /admin y muestra información general del sistema. Incluye **gráficas de adherencia** por médico y por cuidador, supervisión de la relación médico-paciente, **listado de dispositivos IoT** y su estado, además del ranking de mejora. Este ranking usa dos gráficas: una compara el porcentaje anterior con el porcentaje reciente, y otra muestra la posición de cada persona según su mejora o disminución en la adherencia.
- **Vistas analíticas detalladas:** Además de los dashboards principales, el sistema tiene vistas más detalladas desde el perfil de cada paciente. Por ejemplo, el médico puede ver la gráfica de tendencia con promedio móvil de 7 días, la serie de tomas en el tiempo, el riesgo de omisión por paciente y el mapa de proximidad con la última ubicación GPS. También existe una vista de historial de proximidad, donde se pueden revisar trayectos completos usando los puntos guardados en MongoDB.

9. Integración PostgreSQL y MongoDB

9.1 Arquitectura híbrida

El sistema combina las ventajas de dos tipos de bases de datos.

PostgreSQL se usa para guardar la información principal del sistema. Ayuda a mantener los datos correctos, relacionados y seguros. También permite usar procedimientos almacenados, triggers y consultas SQL más completas para analizar información.

MongoDB se usa como apoyo. Sirve para guardar datos más flexibles, como documentos, logs, históricos para gráficas y datos preparados para mostrarse más rápido en la interfaz. También permite borrar ciertos registros automáticamente después de un tiempo, como en el caso de algunos logs.

9.2 Flujo de datos

Existen tres flujos principales de escritura en el sistema, y cada uno funciona de forma diferente.

- **Flujo de eventos NFC:** Cuando el cuidador escanea una etiqueta NFC, Flask ejecuta el procedimiento **sp_registrar_toma_nfc**. Si el registro sale bien, después se ejecuta **sync_evento_nfc** en el archivo **mongo_client.py**. Esta función guarda o actualiza el evento en MongoDB dentro de la colección **eventos_nfc_rt**, usando el identificador **pg_id_evento**. Si el UID de la etiqueta no existe o no está registrado, se llama al procedimiento **sp_nfc_escaneo_desconocido**. Al mismo tiempo, la función **registrar_log_nfc_fallido** guarda el intento fallido en MongoDB, dentro de la colección **logs_nfc_fallidos**, indicando el motivo del error.
- **Flujo GPS doble:** El sistema puede recibir la ubicación GPS de dos formas. La primera es mediante **Traccar Client**, que manda la ubicación al endpoint **/api/gps**. Esta información se guarda en la tabla **ubicacion_gps** de PostgreSQL y también en la colección **historial_gps** de MongoDB. La segunda forma es usando la ubicación del navegador con **navigator.geolocation**. Esta información llega al endpoint **/api/ubicacion-gps** y se guarda en MongoDB mediante la función **agregar_ubicacion_gps**, dentro de la colección **ubicaciones_gps_hist**. Tener estas dos fuentes permite saber de dónde viene cada punto de ubicación y comparar los datos cuando se necesita revisar si el cuidador estuvo cerca del domicilio del paciente.
- **Flujo de logs y errores:** Cada inicio y cierre de sesión se guarda tanto en **log_acceso** de PostgreSQL como en **logs_acceso** de MongoDB. Los errores detectados en Flask se guardan en **logs_sistema**. Estas funciones están preparadas para no detener el sistema si MongoDB falla. Esto ayuda a que un problema en los logs o registros secundarios no afecte la operación clínica principal.

- **Flujo de lectura:** La lectura de datos se hace dependiendo del tipo de información. Las gráficas del cuidador y los trayectos GPS leen directamente desde MongoDB, porque son datos grandes y se consultan con frecuencia. En cambio, los reportes más complejos, como tendencia, riesgo, ranking y supervisión, siguen usando PostgreSQL, porque necesitan cruzar muchas tablas y PostgreSQL lo maneja mejor. El contador de alertas del menú se consulta desde la colección **alertas_rt**, para obtener el dato de forma rápida con una sola consulta.

9.3 Justificación técnica

La decisión de usar un modelo combinado, en lugar de usar solamente PostgreSQL o solamente MongoDB, se basa en cuatro razones principales.

Primero, el sistema maneja diferentes tipos de datos. La información clínica, como pacientes, médicos, recetas y agendas, necesita estar bien organizada, relacionada y protegida con reglas claras. Por eso, **PostgreSQL** es la mejor opción para esta parte. En cambio, la información de apoyo, como notas clínicas, logs, datos preparados para gráficas y recorridos GPS, funciona mejor en **MongoDB**, porque permite guardar datos de forma más flexible.

Segundo, MongoDB ayuda a que las gráficas carguen más rápido. Las gráficas de Highcharts se muestran cuando el médico o cuidador entra al dashboard o cambia de paciente. Si esos datos se calcularán cada vez usando muchas uniones entre tablas de PostgreSQL, el sistema podría tardar más. Por eso, se guarda una colección llamada **historico_adherencia**, donde ya están preparados los datos por día y por paciente. Así, la información se obtiene con una sola consulta y de forma más rápida.

Tercero, MongoDB permite borrar ciertos registros automáticamente después de un tiempo. Esto es útil para los logs de acceso y los logs del sistema. Por ejemplo, algunos registros se eliminan después de **90 días** y otros después de **30 días**, sin necesidad de crear procesos extra para limpiarlos.

Cuarto, el proyecto también cumple con el requisito académico de usar una base de datos relacional y una base NoSQL. Pero esta integración no se hizo solo por cumplir. Cada base tiene una función clara: **PostgreSQL** guarda la información principal y más importante, mientras que **MongoDB** apoya con datos flexibles, rápidos y de mayor volumen.

10. Resultados

10.1 Impacto del sistema

El sistema logró todas sus metas funcionando en tiempo real para detectar al instante si las medicinas se dan tarde, repetidas o se olvidan, además de crear los horarios automáticamente sin tener que escribirlos a mano. También garantiza que el cuidador esté físicamente con el paciente usando el celular para registrar el lugar y la hora, rastreando su ubicación todo el tiempo sin que el empleado tenga que hacer nada extra. Una de sus mejores ventajas es que sabe diferenciar entre la salud y el trabajo: si el paciente tomó bien su pastilla, lo marca como un éxito médico, pero si el cuidador no estaba en la casa en ese momento, le manda una alerta al jefe para que revise la situación. Finalmente, convierte toda esta información en reportes muy claros que muestran fácilmente qué pacientes están mejorando y cuáles corren el riesgo de empeorar.

10.2 Beneficios obtenidos

Este sistema beneficia a todos de manera muy práctica. El paciente recibe un cuidado más seguro porque se evitan errores, olvidos o darle la misma medicina dos veces. El cuidador tiene una herramienta muy fácil de usar que le muestra sus tareas del día y anota su ubicación automáticamente con solo acercar el celular. Por su parte, el doctor puede ver todo el historial del paciente para darle mejores tratamientos y confirmar en un mapa que el empleado realmente estaba en la casa a la hora de darle la pastilla. Además, el administrador puede ver fácilmente qué empleados hacen mejor su trabajo y revisar que todos los aparatos funcionen bien. Finalmente, para los programadores, el sistema quedó tan ordenado que será muy fácil de cuidar, arreglar y mejorar en el futuro.

11. Conclusiones

11.1 Aprendizajes

Crear este sistema fue una gran lección sobre cómo hacer programas útiles para la salud. Aprendimos a guardar las reglas más importantes directamente en la base de datos, lo que al principio fue raro, pero resultó muy bien porque nos permitió cambiar detalles médicos sin tener que modificar el resto del código. También descubrimos que una página web es mucho más que texto y botones.

Aprendimos a conectar herramientas del celular, como los mapas y los lectores de chips, para que trabajen en equipo sin fallar. Además, logramos que el sistema pudiera recibir y entender diferentes tipos de información de ubicación sin confundirse, y entendimos cómo usar distintos tipos de bases de datos. En unas

organizas todo en tablas estrictas y en otras simplemente guardas la información tal como la vas a querer leer después. Por último, nos dimos cuenta de que no siempre hay que usar las herramientas más complejas o "de moda", sino que es mejor elegir las opciones más prácticas que te permitan enfocarte en que el proyecto realmente funcione.

11.2 Limitaciones

Entre sus limitaciones técnicas, la función para escanear las medicinas solo sirve en celulares Android usando Google Chrome, por lo que no funciona en iPhones. Además, para comprobar si el cuidador está en la casa usamos el GPS del celular, en lugar de conectar sensores de proximidad reales. En cuanto a la seguridad, las contraseñas están guardadas de forma un poco básica, algo que funciona para el proyecto académico pero que no sería seguro en el mundo real. También, las gráficas o íconos de la pantalla podrían fallar de repente si sus creadores originales los modifican en internet.

Por último, son los propios cuidadores quienes eligen en su aplicación cada cuánto tiempo mandan su ubicación, lo que hace que la información llegue de forma dispareja en lugar de tener un control automático e igual para todos.

11.3 Trabajo futuro

Las siguientes mejoras serían buenas opciones para seguir desarrollando MediNFC en el futuro.

- **Crear una aplicación móvil para cuidadores**, que pueda usar NFC y Bluetooth de forma más estable en Android e iOS. Esto ayudaría a dejar de depender de Chrome en Android y permitiría escanear beacons reales por Bluetooth.
- **Agregar una opción para escribir manualmente el UID**, en caso de que el navegador no pueda leer etiquetas NFC. También se podría crear un endpoint REST para que una lectora física pueda mandar las lecturas directamente al backend.
- **Mover las contraseñas y datos sensibles a variables de entorno**, usando un archivo .env. También se podría agregar una opción para cambiar contraseñas y manejar tokens API desde el panel de administrador.

- **Agregar notificaciones push y SMS reales** usando un proveedor externo. Actualmente el sistema registra el canal de alerta, pero no envía físicamente el mensaje al usuario.
- **Separar el scheduler del proceso principal** o cambiarlo a Celery con Redis. Esto ayudaría a evitar que los procesos automáticos se dupliquen cuando se use el sistema con varios workers.
- **Definir tiempos de eliminación automática en MongoDB** para todas las colecciones, según el tipo y cantidad de datos que se espera guardar.
- **Crear un modelo predictivo de riesgo**, usando el historial de tomas, alertas y resultados clínicos. Así, el médico podría ver una predicción de riesgo a 7 o 14 días, en lugar de solo revisar datos pasados.
- **Conectar el sistema con expedientes clínicos electrónicos**, usando estándares como HL7 FHIR.
- **Dividir las tablas de eventos por periodos de tiempo** para mejorar el rendimiento cuando haya muchos datos. También se podría mejorar la sincronización con MongoDB usando colas y procesos en segundo plano.
- **Hacer una prueba clínica formal con un grupo piloto**, para validar el sistema en un caso real y ajustar los niveles de riesgo y tendencia con base en evidencia real.

12. Referencias

Mozilla Developer Network. (2025). Web NFC API. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_NFC_API

Traccar Ltd. (2026). Traccar — GPS tracking software: Free and open source system. <https://www.traccar.org/documentation/>

Traccar Ltd. (2026). OsmAnd protocol. <https://www.traccar.org/osmand/>

Sabaté, E. (Ed.). (2003). Adherence to long-term therapies: Evidence for action. World Health Organization. <https://iris.who.int/items/bf8058c0-03b2-4b47-838f-5534849927fb>

The PostgreSQL Global Development Group. (2026). PL/pgSQL — SQL Procedural Language (v. 18). <https://www.postgresql.org/docs/current/plpgsql.html>

The PostgreSQL Global Development Group. (2026). CREATE PROCEDURE (v. 18). <https://www.postgresql.org/docs/current/sql-createprocedure.html>

MongoDB, Inc. (2026). TTL indexes. <https://www.mongodb.com/docs/manual/core/index-ttl/>

Pallets Projects. (2025). Quickstart — Flask documentation (3.1.x). <https://flask.palletsprojects.com/en/stable/quickstart/>

Agafonkin, V. (2023). Leaflet 1.9.4 — API reference. <https://leafletjs.com/reference.html>

Provos, N., & Mazières, D. (1999). A future-adaptable password scheme. En Proceedings of the 1999 USENIX Annual Technical Conference, FREENIX Track (pp. 81–91). USENIX Association. https://www.usenix.org/legacy/events/usenix99/provos/provos_html/